N

Optimizing a Feed Forward Neural Network

We will consider the case in which the FFNN is a classifier function

$$f(x;w) : \mathbb{R}^p \to \mathbb{R}$$

Where we say that input $x \in \mathbb{R}^p$ belongs to class 0 or class 1 if $f(x;w) < c$ or $f(x;w) \geq c$, respectively. Here $w \in \mathbb{R}^n$ is a parameter vector defining the structure of the FFNN.

A not very practical example:

$$f(x;w) = Wx \quad \text{with } c=5$$

where $x \in \mathbb{R}^p$ and W is a $1 \times p$ matrix.

Let $p=3$ and $W = [2 \ 1 \ 4]$ Then the classifier is

$$f(x) = [2 \ 1 \ 4] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = 2x_1 + x_2 + 4x_3$$

For $x^T = [1 \ 0 \ 1]$, $f(x) = 2+4 = 6$, x is in class 1

For $x^T = [1 \ 1 \ 1]$, $f(x) = 2+1+4 = 7$, x is in class 1

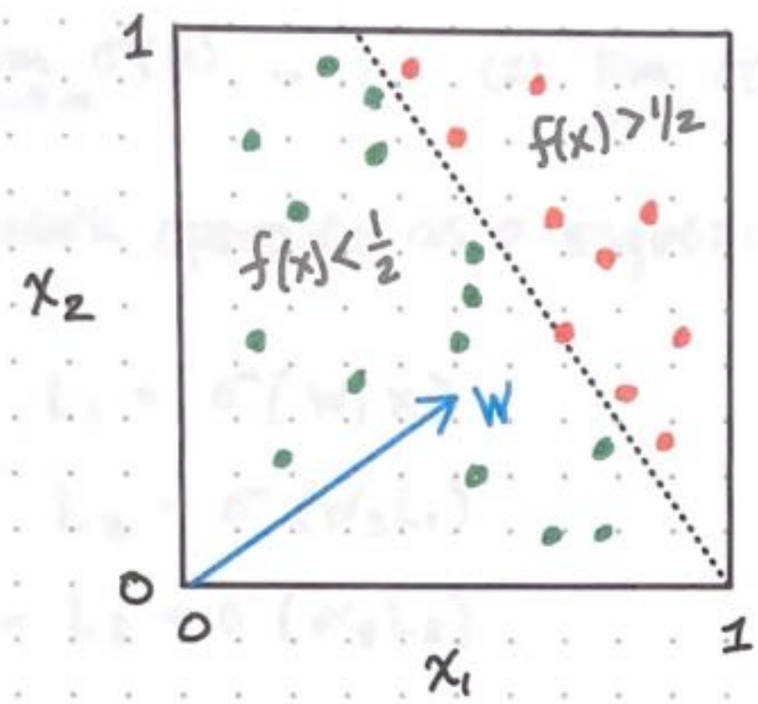
For $x^T = [-1 \ 1 \ 1]$, $f(x) = -2+1+4 = 3$, x is in class 0

The general optimization problem is to find an optimal set of parameters w so that $f(x;w)$ correctly classifies a known data set (the training set). The hope is that this same optimized f will also correctly classify data which was not used in training.

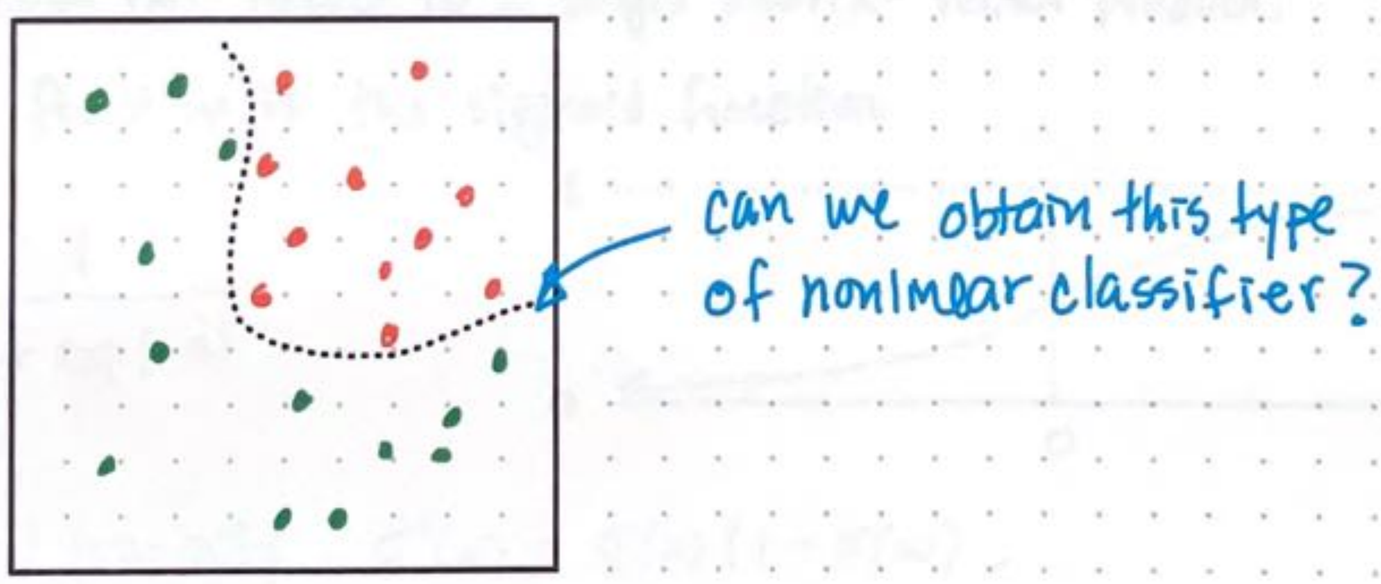
A slightly more involved example: $f(x;w) = W_3(W_2(W_1 x))$ where W_1 is a $k_1 \times n$ matrix, W_2 is a $k_2 \times k_1$ matrix, W_3 is a $1 \times k_2$ matrix. The weights $w \in \mathbb{R}^n$ are the numerical entries in the W matrices, collectively. This classifier is a composition of three transformations. But on close inspection is equivalent to the first example $f(x,w) = Wx$ with $W = W_1 W_2 W_3$.

(here $n = k_1 n + k_2 k_1 + k_2$)

The previous examples are linear classifiers which have a nice visual interpretation. Suppose we have the linear classifier $f(x) = [\,1/2 \;\; 1/3\,]\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ where $x \in [0,1]^2$. $(w = [\,1/2 \;\; 1/3\,])$.



But real data is not always separable by a linear function.



A simple feed-forward neural network is a specific type of nonlinearization of the previous example.

$$f(x;w) = \sigma(W_3\, \sigma(W_2 \sigma(W_1 x)))$$

where $\sigma(u)$ is an elementwise function with three properties:

(1) $\lim_{u\to\infty} \sigma(u) = 1$ (2) $\lim_{u\to-\infty} \sigma(u) = 0$ (3) $\sigma'(u) > 0$

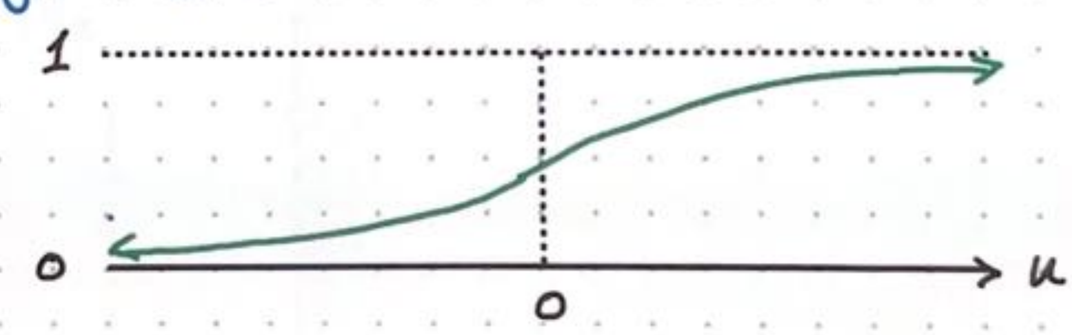
The network operates as a sequence of functions.

$$L_1 = \sigma(W_1 x)$$
$$L_2 = \sigma(W_2 L_1)$$
$$f = L_3 = \sigma(W_3 L_2)$$

This example does not reduce to a single matrix-vector product.

A typical σ function is the sigmoid function

$$\sigma(u) = \frac{1}{1+\exp(-u)}$$



with the useful property $\sigma'(u) = \sigma(u)(1-\sigma(u))$.

Altogether we have $f: \mathbb{R}^p \to (0,1)$ with cutoff $c = 1/2$

Example: $f(x;\omega) = \sigma(W_3\sigma(W_2\sigma(W_1 x)))$ where $x \in \mathbb{R}^2$

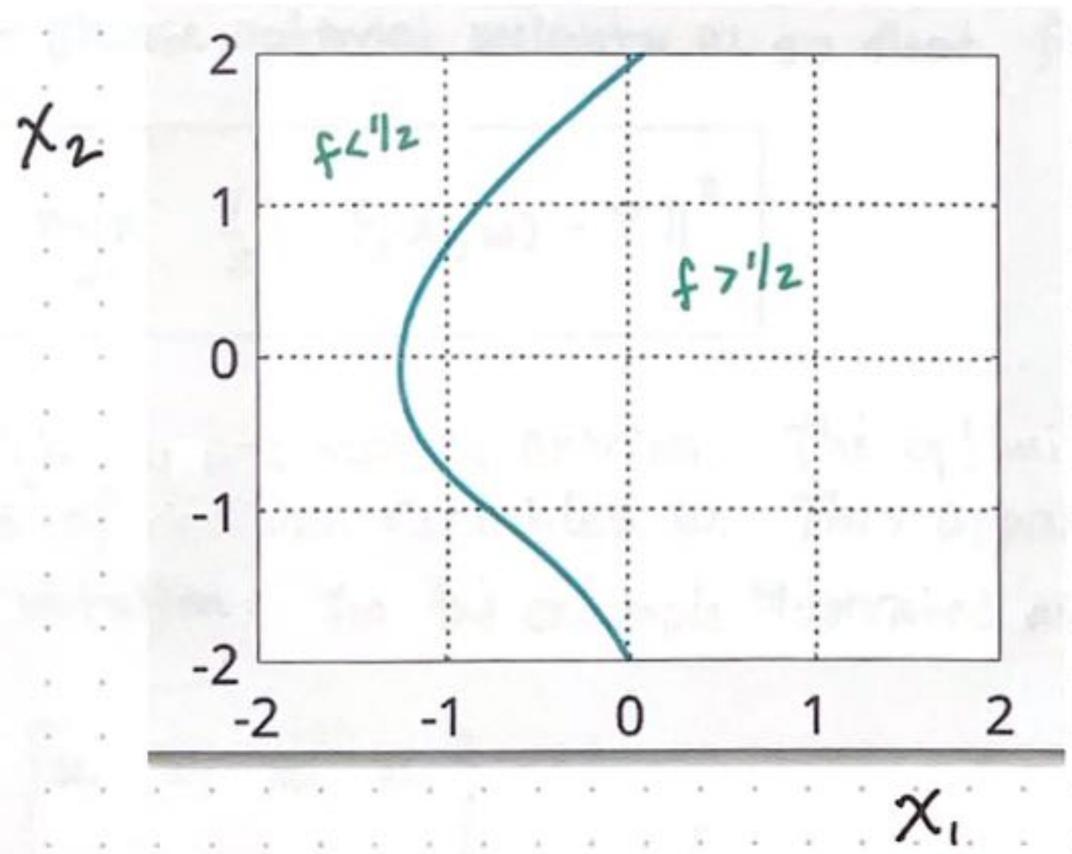
$$W_1 = \begin{bmatrix} -1 & 1 \\ 0 & 1 \\ 1/2 & -1 \end{bmatrix} \quad W_2 = \begin{bmatrix} 1/2 & 1/3 & 0 \\ -1/3 & 1/4 & 1 \\ 1 & 1 & -1 \end{bmatrix} \quad W_3 = \begin{bmatrix} -1 & 1/2 & 1/2 \end{bmatrix}$$

For $x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ $\quad W_1 x = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$ $\quad \sigma(W_1 x) = \begin{bmatrix} 0.269 \\ 0.731 \\ 0.500 \end{bmatrix} \equiv L_1$

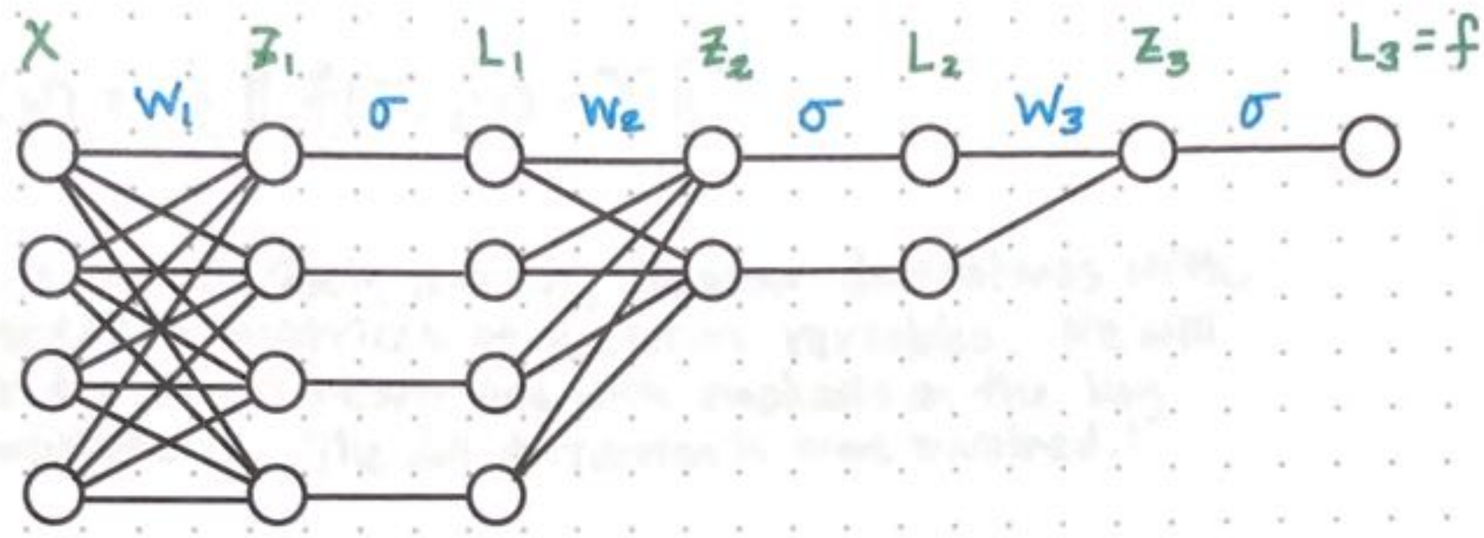
$$W_2 L_1 = \begin{bmatrix} 0.378 \\ 0.593 \\ 0.500 \end{bmatrix} \quad \sigma(W_2 L_1) = \begin{bmatrix} 0.593 \\ 0.644 \\ 0.622 \end{bmatrix} \equiv L_2$$

$$W_3 L_2 = 0.673 \quad \sigma(W_3 L_2) = 0.662 = f$$

$f \geq 1/2 \Rightarrow x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ belongs to class 1.



We can represent a FFNN as a left-to-right computational network. The previous example may be represented as follows:



In this example, W_1 is 4×4, W_2 is 2×4, W_3 is 1×2.

That makes $16+8+2 = 26$ weights in the W matrices. Each choice of $w \in \mathbb{R}^{26}$ results in a different function $f(x;w)$.

Now, suppose we have "training data" $X \in \mathbb{R}^{p\times n}$ where each column is a "measurement" or an example $x \in \mathbb{R}^p$, and we have a corresponding class vector $Y \in \{0,1\}^n$. We can use this data (X,Y) to "train" the FFNN — choose optimal weights w so that $f(x_k;w) \approx y_k$ for all k.

$$\boxed{\min_{w} \ \frac{1}{2}\| f(X;w) - Y \|^2}$$

The weights w are matrix entries. The optimization code operates on a vector of decision variables w. This apparent discrepancy is only one of notation. In the example illustrated as the network, we have:

$$W_1 = \begin{bmatrix} w_1 & w_5 & w_9 & w_{13} \\ w_2 & w_6 & w_{10} & w_{14} \\ w_3 & w_7 & w_{11} & w_{15} \\ w_4 & w_8 & w_{12} & w_{16} \end{bmatrix}, \quad W_2 = \begin{bmatrix} w_{17} & w_{19} & w_{21} & w_{23} \\ w_{18} & w_{20} & w_{22} & w_{24} \end{bmatrix}, \quad W_3 = \begin{bmatrix} w_{25} & w_{26} \end{bmatrix}$$

and $w \in \mathbb{R}^{26}$

The gradient of the objective is computed almost as easily as the objective itself.

$$g(\omega) = \frac{1}{2} \| f(X;\omega) - Y \|^2$$

(While not precisely correct, consider derivatives with respect to matrices of decision variables. We will get the correct result and with emphasis on the key computations. The full derivation is more involved.)

$$g(\omega) = \frac{1}{2} \| f(X;\omega) - Y \|^2$$

$$\frac{\partial g}{\partial W_3} = (L_3 - Y) \frac{\partial L_3}{\partial W_3}$$

$$= (L_3 - Y) \frac{\partial}{\partial W_3} \sigma(W_3 L_2)$$

chain rule!

$$= (L_3 - Y) \sigma'(W_3 L_2) L_2^T$$

$$= (L_3 - Y) \sigma(W_3 L_2)(1 - \sigma(W_3 L_2)) L_2^T$$

$$= \underbrace{(L_3 - Y) L_3 (1 - L_3)}_{\text{element-wise products}} L_2^T$$

Altogether:

forward	backward	
$L_1 = \sigma(W_1 X)$	$h_d = (L_d - Y) L_d (1 - L_d)$	$G_d = h_d L_{d-1}^T$
$L_2 = \sigma(W_2 L_1)$	$h_{d-1} = W_d^T h_d L_{d-1} (1 - L_{d-1})$	$G_{d-1} = h_{d-1} L_{d-2}^T$
$\vdots$	$\vdots$	$\vdots$
$L_d = \sigma(W_d L_{d-1})$	$h_2 = W_3^T h_3 L_2 (1 - L_2)$	$G_2 = h_2 L_1^T$
$f = \frac{1}{2} \| L_d - Y \|^2$	$h_1 = W_2^T h_2 L_1 (1 - L_1)$	$G_1 = h_1 X^T$

The elements of G_i are the derivatives of $g(\omega)$ with respect to the elements of W_i.

Once we have optimized (trained) our FFNN, the next task is to test the ability of the FFNN to correctly classify new data (at least data which was not used in the training phase).

Full data set

N instances of $x \in \mathbb{R}^p$.

X is $p \times N$

Training Data Set

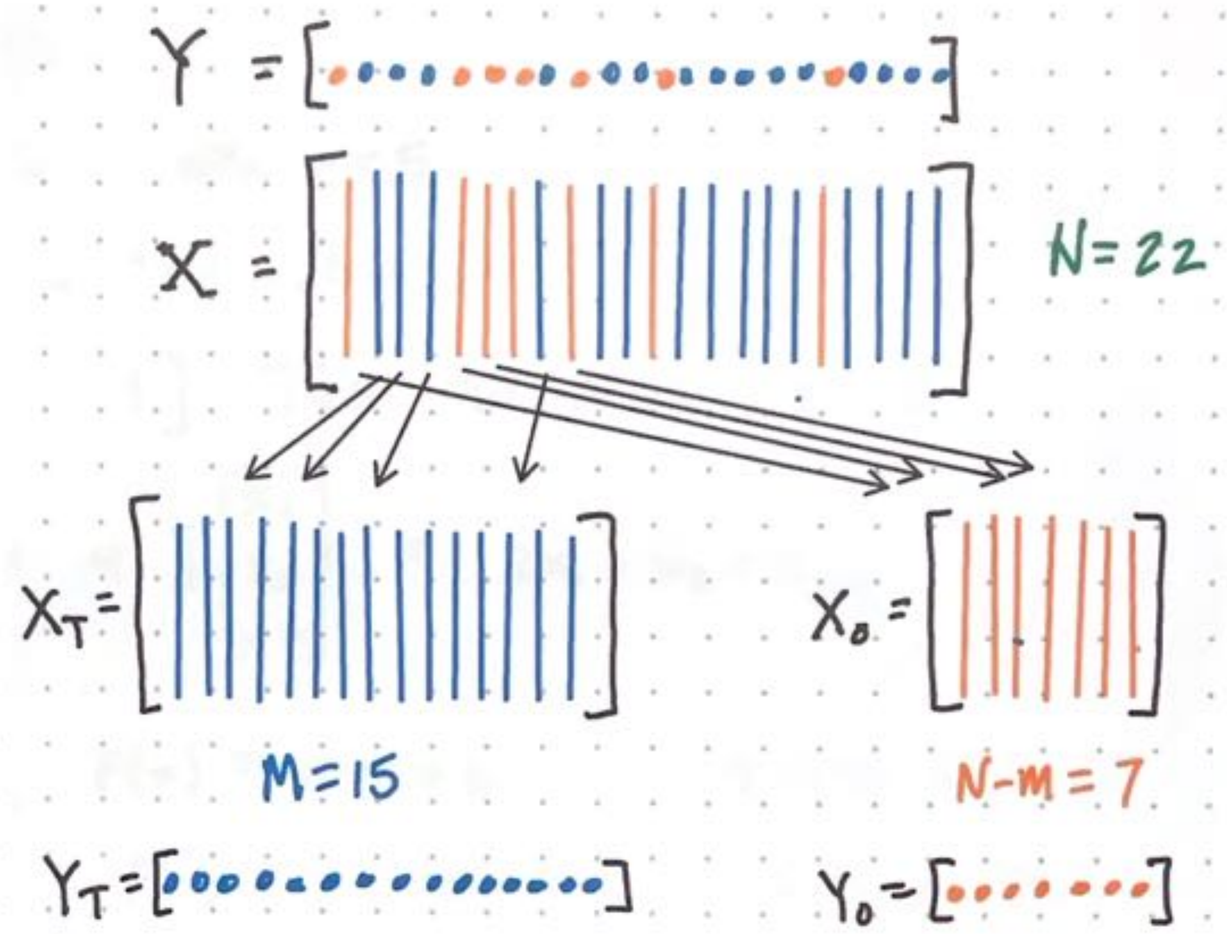
$M < N$ instances

X_T is $p \times M$

Testing Data Set

$N-M$ instances

X_0 is $p \times (N-M)$



X_T is used to build the FFNN (determine w^*).

Let x_k be the kth column of X_0, and y_k the kth entry of Y_0.

A correct classification is one for which $\text{round}(f(x_k; w^*)) = y_k$.

2×2 Confusion Matrix M

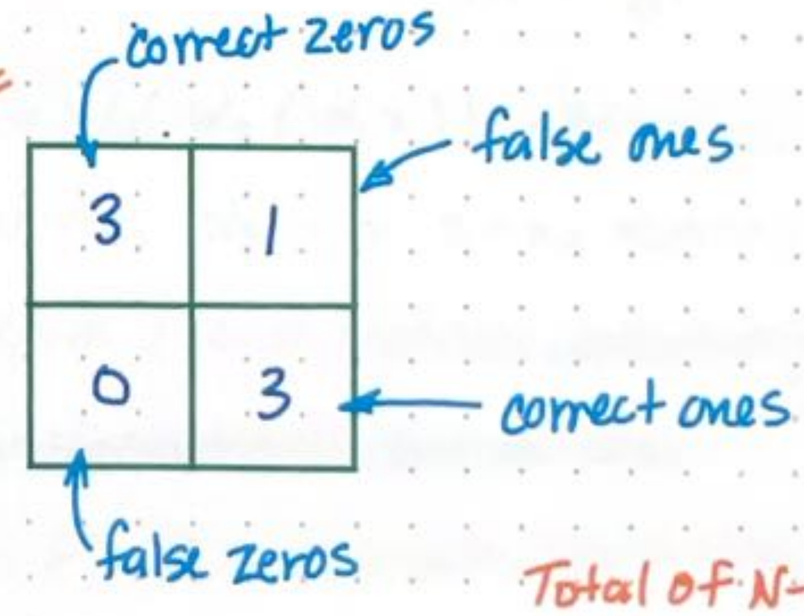
$M_{11} = \#(y_k = 0 \text{ and } f_k = 0)$

$M_{12} = \#(y_k = 0 \text{ and } f_k = 1)$

$M_{21} = \#(y_k = 1 \text{ and } f_k = 0)$

$M_{22} = \#(y_k = 1 \text{ and } f_k = 1)$

example



Total of $N-M=7$ test data instances